GL Workflow Pipeline Architecture

Full Metadata Schema, Angular Frontend Architecture & Airflow 3.x Orchestration

1. SYSTEM ARCHITECTURE OVERVIEW

The GL Workflow Pipeline Engine is a metadata-driven orchestration framework designed to move business logic out of hardcoded DAG files and into a relational database. It enables dynamic user creation, sequencing, and specification linking via an Angular UI while maintaining lean Airflow 3.x DAG execution wrappers.

Core Capabilities

- **Independent Reference Entities:** Pre-defined data sources (e.g., `FXO`, `POWER`, `GECD`) and standard component types (`data_transfer`, `exclusion_type`, `filter_type`, `control_type`, `pipeline`, `logical_component`).
- **Global Logical Components:** Generic reusable components independent of data sources (represented by `data_source_id IS NULL`) available to all pipelines.
- **Dynamic Task Function Chains:** Components group fine-grained tasks executing across `pre`, `main`, and `post` phases with execution priorities and parallel execution support.
- **Task-Level Column Transformations:** Detailed source-to-destination table and field-level mappings with embedded transformation rules (e.g., `to_upper`, `trim`) assigned directly to main tasks.
- **Nested Pipelines & Branching:** Supports child pipeline components (`nested_pipeline_id`) and edge-based routing for component-level flow and task-level failure recovery.
- **Traceability & Spec Links:** Full documentation linkage (Functional Specs, Technical Specs, docstrings, and justification notes) across every level.

2. RELATIONAL DATABASE DDL SCHEMA (PART 1)

The DDL below defines the master reference structures, data source component maps (supporting generic logical components via NULL data sources), task chains, and pipeline master schedules.

```
-- 1. Reference: Data Sources Master
CREATE TABLE data_sources (
    data_source_id INT PRIMARY KEY,
    data_source_name VARCHAR(50) NOT NULL UNIQUE,
    description VARCHAR(250),
    is_active CHAR(1) DEFAULT 'Y',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Reference: Component Types (Includes Generic Logical Component)
CREATE TABLE component_types (
    component_type_id INT PRIMARY KEY,
    component_type_name VARCHAR(50) NOT NULL UNIQUE, -- 'data_transfer', 'exclusion_type', 'filter_
type', 'control_type', 'pipeline', 'logical_component'
    description VARCHAR(250),
    is_active CHAR(1) DEFAULT 'Y',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. Association: Data Source Component Types (data_source_id IS NULL for generic logical componen
ts)
CREATE TABLE data_source_components (
    ds_component_id INT PRIMARY KEY,
    data_source_id INT NULL, -- NULL indicates global component usable across ALL data sources
    component_type_id INT NOT NULL,
    is_active CHAR(1) DEFAULT 'Y',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_dsc_datasource FOREIGN KEY (data_source_id) REFERENCES data_sources(data_source_i
d),
    CONSTRAINT fk_dsc_componenttype FOREIGN KEY (component_type_id) REFERENCES component_types(comp
onent_type_id),
    CONSTRAINT uk_datasource_component UNIQUE (data_source_id, component_type_id)
);

-- 4. Default Available Tasks (Pre, Main, Post functions)
CREATE TABLE data_source_component_tasks (
    task_id INT PRIMARY KEY,
    ds_component_id INT NOT NULL,
    phase VARCHAR(20) NOT NULL,            -- 'pre', 'main', 'post'
    task_name VARCHAR(150) NOT NULL,
    description VARCHAR(250),
    sequence_number INT NOT NULL,
    priority INT DEFAULT 0,
    run_in_parallel CHAR(1) DEFAULT 'N',
    trigger_rule VARCHAR(50) DEFAULT 'all_success',
    is_active CHAR(1) DEFAULT 'Y',
    functional_spec_url VARCHAR(500) NULL,
    technical_spec_url VARCHAR(500) NULL,
    task_docstring TEXT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_dsct_ds_component FOREIGN KEY (ds_component_id) REFERENCES data_source_components
(ds_component_id)
);
```

3. RELATIONAL DATABASE DDL SCHEMA (PART 2)

Tables defining pipelines, schedules, component runtime instances, branching edges, and field-level transformations.

```
-- 5. Pipelines Master Table
CREATE TABLE pipelines (
    pipeline_id INT PRIMARY KEY,
    pipeline_name VARCHAR(150) NOT NULL,
    data_source_id INT NOT NULL,
    description VARCHAR(500),
    is_active CHAR(1) DEFAULT 'Y',
    functional_spec_url VARCHAR(500) NULL,
    technical_spec_url VARCHAR(500) NULL,
    business_justification TEXT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_pipeline_datasource FOREIGN KEY (data_source_id) REFERENCES data_sources(data_sou
rce_id)
);

-- 6. Pipeline Schedules (1-to-1 Relationship)
CREATE TABLE pipeline_schedules (
    pipeline_id INT PRIMARY KEY,
    schedule_type VARCHAR(30) NOT NULL,    -- 'cron', 'interval', 'manual'
    cron_expression VARCHAR(100) NULL,
    interval_seconds INT NULL,
    timezone VARCHAR(50) DEFAULT 'UTC',
    is_schedule_active CHAR(1) DEFAULT 'Y',
    CONSTRAINT fk_pipeline_schedule FOREIGN KEY (pipeline_id) REFERENCES pipelines(pipeline_id) ON
DELETE CASCADE
);

-- 7. Pipeline Canvas Component Instances
CREATE TABLE pipeline_components (
    component_instance_id INT PRIMARY KEY,
    pipeline_id INT NOT NULL,
    ds_component_id INT NOT NULL,
    sequence_number INT NOT NULL,
    nested_pipeline_id INT NULL,           -- Populated if component_type = 'pipeline'
    component_alias VARCHAR(150) NULL,
    is_active CHAR(1) DEFAULT 'Y',
    functional_spec_url VARCHAR(500) NULL,
    technical_spec_url VARCHAR(500) NULL,
    component_docstring TEXT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_pc_pipeline FOREIGN KEY (pipeline_id) REFERENCES pipelines(pipeline_id) ON DELETE
CASCADE,
    CONSTRAINT fk_pc_ds_component FOREIGN KEY (ds_component_id) REFERENCES data_source_components(d
s_component_id),
    CONSTRAINT fk_pc_nested_pipeline FOREIGN KEY (nested_pipeline_id) REFERENCES pipelines(pipeline
_id)
);

-- 8. Component-Level Flow Edges
CREATE TABLE pipeline_component_edges (
    component_edge_id INT PRIMARY KEY,
    pipeline_id INT NOT NULL,
    source_component_instance_id INT NOT NULL,
    target_component_instance_id INT NOT NULL,
    condition_expression VARCHAR(500) NULL,
    condition_type VARCHAR(30) DEFAULT 'always',
    CONSTRAINT fk_pce_pipeline FOREIGN KEY (pipeline_id) REFERENCES pipelines(pipeline_id) ON DELET
E CASCADE,
    CONSTRAINT fk_pce_source FOREIGN KEY (source_component_instance_id) REFERENCES pipeline_compone
nts(component_instance_id) ON DELETE CASCADE,
    CONSTRAINT fk_pce_target FOREIGN KEY (target_component_instance_id) REFERENCES pipeline_compone
nts(component_instance_id) ON DELETE CASCADE
);

-- 9. Task-Level Branching Edges
CREATE TABLE component_task_edges (
```

4. ANGULAR 18 FRONTEND DATA MODELS & COMPONENT LOGIC

Frontend TypeScript interfaces and canvas drop handlers for workflow visual editing.

```
// Data Models (workflow.model.ts)
export interface DataSource {
  dataSourceId: number;
  dataSourceName: string;
}

export interface ComponentType {
  componentTypeId: number;
  componentTypeName: string;
}

export interface PipelineComponentInstance {
  componentInstanceId: string;
  dsComponentId: number;
  componentTypeName: string;
  componentAlias: string;
  sequenceNumber: number;
  nestedPipelineId?: number | null;
  functionalSpecUrl?: string;
  technicalSpecUrl?: string;
  componentDocstring?: string;
  tableMappings?: TableMapping[];
}

export interface TableMapping {
  tableMappingId: string;
  sourceTable: string;
  destinationTable: string;
  loadType: string;
  fieldMappings: FieldMapping[];
}

export interface FieldMapping {
  sourceField: string;
  destinationField: string;
  transformationFunction?: string;
  ruleJustification?: string;
}
```

5. AIRFLOW 3.X TASKFLOW DYNAMIC DAG FACTORY

The generic Airflow DAG builder queries database metadata at runtime to dynamically create DAG tasks.

```
from airflow.decorators import dag, task
from datetime import datetime

# Central application layer entrypoint
python_app_layer = __import__("your_app.engine")

def execute_task_via_metadata(task_id, **context):
    """Generic execution callable invoking core python application layer."""
    python_app_layer.run_task_using_metadata(task_id=task_id, airflow_context=context)

def create_dynamic_pipeline_dag(pipeline_id, dag_id, db_session):
    pipeline_meta = db_session.get_pipeline_definition(pipeline_id)

    @dag(
        dag_id=dag_id,
        start_date=datetime(2026, 1, 1),
        schedule=pipeline_meta.get("cron_expression"),
        catchup=False,
        tags=[pipeline_meta["data_source_name"], f"pipeline_{pipeline_id}"]
    )
    def generated_dag():
        last_task_reference = None

        for comp in pipeline_meta["components"]:
            for task_meta in comp["tasks"]:
                t_id = task_meta["task_id"]
                phase = task_meta["phase"]
                t_name = task_meta["task_name"]

                airflow_task_id = f"comp_{comp['component_instance_id']}_{phase}_{t_name}"

                @task(task_id=airflow_task_id, trigger_rule=task_meta.get("trigger_rule", "all_succ
ess"))
                def run_app_wrapper(id_to_run=t_id, **kwargs):
                    execute_task_via_metadata(id_to_run, **kwargs)

                task_instance = run_app_wrapper()

                if last_task_reference:
                    last_task_reference >> task_instance
                last_task_reference = task_instance

    return generated_dag()
```